

HAYVAN SAHİPLENDİRME PLATFORMU

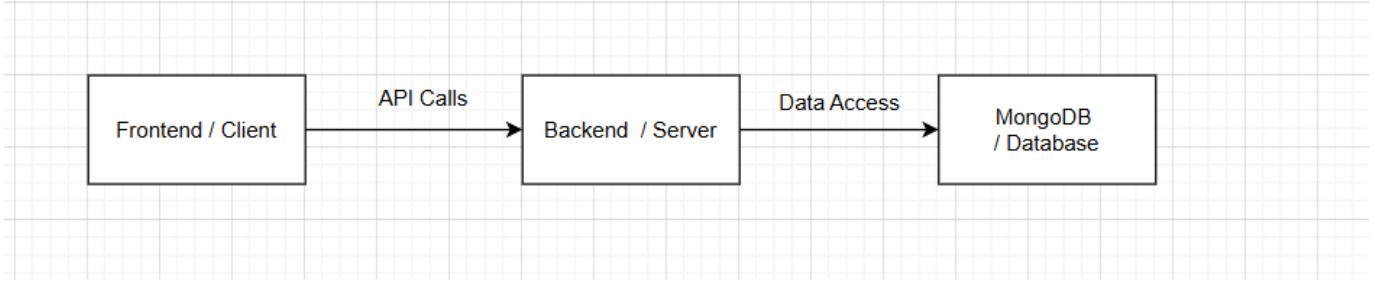
1. GİRİŞ

Bu rapor, "Hayvan Sahiplendirme Platformu" adlı sosyal sorumluluk projesi kapsamında geliştirilecek olan web tabanlı yazılımın mimari tasarımını açıklamaktadır. Projenin temel amacı; ekonomik sebeplerle ya da çeşitli nedenlerle sahiplendirilecek evcil hayvanların yeni yuvalarına güvenli bir şekilde ulaşmasını sağlamak, kaybolan hayvanların bulunmasına yardımcı olmak ve kullanıcıların bu süreçte organize bir biçimde etkileşimde bulunabileceği dijital bir ortam oluşturmaktır.

Bu doğrultuda geliştirilecek olan sistem, kullanıcıların ilan oluşturabildiği, arama filtreleriyle ihtiyaç duyduğu hayvana veya ilana kolayca ulaşabildiği, kullanıcı geri bildirimleriyle güven ortamının sağlandığı, ayrıca veteriner ve barınak gibi kurumların da entegre edilebildiği çok yönlü bir yapıya sahip olacaktır.

Mimari tasarım raporunda; sistemin alt bileşenleri, sınıf yapıları, arayüzler, rutinler, kullanılacak teknolojiler, kodlama standartları ve test stratejileri ayrıntılı olarak açıklanmıştır. Bu doküman, yazılım mühendisliği süreçlerine uygun olarak, sistemin sürdürülebilir ve geliştirilebilir bir mimariyle inşa edilmesini sağlamak amacıyla hazırlanmıştır.

2. SİSTEM MİMARİSİ

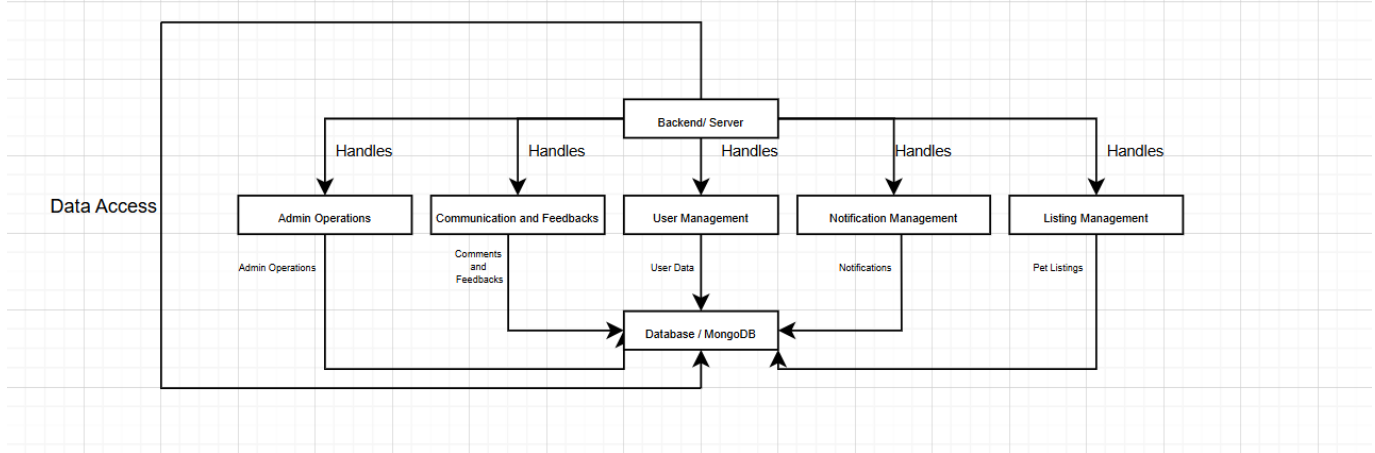


Hayvan Sahiplendirme Platformu, modern web teknolojileri kullanılarak geliştirilmiş full-stack bir web uygulamasıdır. Sistem, istemci (frontend) ve sunucu (backend) olmak üzere iki ana bileşenden oluşur. Bu iki katman birbirleriyle RESTful API üzerinden iletişim kurar. Uygulama, ölçeklenebilirlik, sürdürülebilirlik ve geliştirilebilirlik esaslarına göre yapılandırılmıştır.

2.1 Mimari Yaklaşım

Platformda MVC (Model-View-Controller) ve modüler mikro yapı benimsenmiştir. Ayrıca, modüler bileşen yapısı ve arayüzler üzerinden ayrılmış kontrol akışları ile sistemde gevşek bağlı (loosely coupled) bir yapı sağlanmıştır.

2.2 Backend Mimarisi

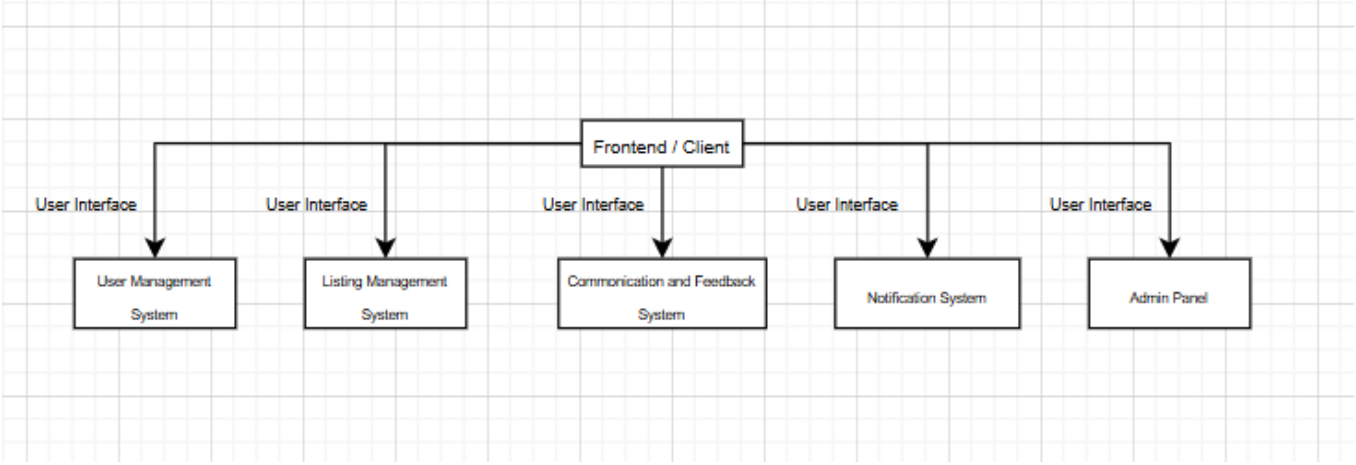


Backend Node.js ortamında Express.js framework'ü kullanılarak geliştirilmiştir. Mongoose kullanılarak MongoDB ile veri etkileşimi sağlanır. Aşağıda backend klasör yapısı ve görevleri özetlenmiştir:

Backend Klasör Yapısı:

- * app.js: Uygulamanın giriş noktası, tüm middleware ve routerleri burada bağlanır.
- * routes/: Tüm API uç noktaları (örneğin: ilan, kullanıcı, yorum vb.) burada tanımlanır.
- * controllers/: Her bir routeye karşılık gelen iş mantığı burada tanımlanır.
- * models/: MongoDB için tanımlanan Mongoose şemaları burada yer alır.
- * middleware/: JWT doğrulama, hata yönetimi gibi özel middleware fonksiyonları.
- * validators/: Gelen isteklerin doğrulaması için kullanılan Zod/Express-validator yapılandırmaları.
- * utils/: Şifreleme, token üretimi, tarih formatlama gibi yardımcı fonksiyonlar.
- * config/: Ortam değişkenleri, veritabanı bağlantısı gibi yapılandırma dosyaları.
- * endpoints/: API endpointlerin sabit tanımları.
- * seed.js: Test verilerini veritabanına eklemek için kullanılan script.
- * Dockerfile, .env.example: Uygulamanın konteynerleştirilmesi ve çevresel değişken örnekleri.

2.3 Frontend Mimarisi



Frontend, Next.js 15 kullanılarak geliştirilmiştir. Proje Tailwind CSS ile style özellikleri verilmiş, Zustand ile global state yönetimi sağlanmıştır. Kod modüller, yeniden kullanılabilir component yapısıyla organize edilmiştir.

Frontend Klasör Yapısı:

- * src/app/: Next.js 15'ün App Router yapısına uygun olarak sayfa ve layout yapısı burada kurulur.
- * src/components/: Single UI bileşenleri
- * src/containers/: Sayfaların birleştirici bileşenleri
- * src/context/: Auth veya tema gibi global context sağlayıcılar.
- * src/middlewares/: Sayfalar arası geçişler ve auth middleware'leri
- * src/hooks/: Özel React hookları
- * src/services/: API isteklerini yöneten servis dosyaları.
- * src/store/: Zustand kullanarak oluşturulan global state yapılandırmaları.
- * mocks/: Mock veriler test amaçlı burada saklanır.
- * eslint.config.mjs, postcss.config.mjs, next.config.js: Geliştirme ve derleme yapılandırma dosyaları.

2.4 İstemci-Sunucu İletişimi

Frontend, backend API'lerine src/services/ altında tanımlanan servisler aracılığıyla HTTP (fetch/axios) üzerinden erişir. JSON tabanlı veri alışverişi sağlanır. Kimlik doğrulama JWT ile gerçekleştirilir. Kullanıcı bilgisi ve token bilgisi Zustand global state'te tutulur.

2.5 Dağıtım (Deployment) ve Ortamlar

Frontend → Vercel kullanılarak dağıtılacaktır.

Backend → Docker konteyneri olarak çalıştırılabilir. MongoDB Atlas kullanılacaktır.

.env.example → Tüm ortam değişkenleri versiyon kontrolüne dahil olmayan .env dosyasıyla sağlanır.

3. ALT SİSTEMLER

Hayvan Sahiplendirme Platformu, farklı işlevleri yerine getirmek üzere birbirinden bağımsız ancak birbirine entegre çalışan alt sistemlerden oluşmaktadır. Bu modüler yapı sayesinde sistem daha esnek, sürdürülebilir ve test

edilebilir bir hale getirilmiştir. Aşağıda her alt sistemin görevi, diğer sistemlerle olan ilişkisi ve genel işlevi açıklanmıştır.

3.1 Kullanıcı Yönetim Sistemi

Bu alt sistem, kullanıcıların platforma kayıt olması, giriş yapması, profil bilgilerini yönetmesi ve yetkilendirme işlemlerinden sorumludur.

- * JWT ile kimlik doğrulama
- * Kullanıcı rolleri: normal kullanıcı, admin
- * Profil bilgileri güncelleme

3.2 İlan Yönetim Sistemi

Kullanıcıların sahiplendirme ya da kayıp ilanlarını oluşturmaya, görüntülemesine, düzenlemesine ve silmesine olanak tanır.

- * Hayvan türü, cinsiyet, yaş, sağlık durumu, açıklama ve konum bilgileri içeren ilan yapısı
- * Kayıp ve sahiplendirme ilanları ayrımı
- * Kullanıcıların favori ilanları listelemesi

3.3 İletişim ve Geri Bildirim Sistemi

Kullanıcılar arasında iletişimi sağlamak ve sistemin kalitesini artırmak için geri bildirim mekanizması sunar.

- * İlan sahipleriyle iletişim (form, mesaj, MVP'den sonra live chat)
- * Kullanıcı puanlama ve değerlendirme
- * Moderatörler için raporlama altyapısı

3.4 Bildirim Sistemi

Kullanıcıların sistemdeki gelişmelerden anlık haberdar olmasını sağlar.

- * Yeni mesaj, başvuru, yorum vb. durumlarda bildirim
- * WebSocket teknolojisiyle gerçek zamanlı iletişim(MVP'den sonra)
- * Bildirim bileşenleri frontend'de gösterilir

3.5 Moderasyon ve Güvenlik Sistemi

Platformda olumsuz kullanıcı davranışlarını engellemek ve sistemi güvenli tutmak için kullanılır.

- * AI destekli içerik denetimi (NLP teknolojileri ile MVP'den sonra)
- * Kurallara aykırı davranışların tespiti
- * Banlama ve uyarı sistemi
- * Güvenlik testleri: JWT, lighthouse

3.6 Yönetici Paneli (Admin Panel)

Platform yöneticilerinin sistem genelinde kontrol sağlayabilmesi için geliştirilmiştir.

- * Kullanıcı ve ilan denetimi
- * Sistem loglarını inceleme
- * Geri bildirimlere erişim
- * İstatistik ve yönetsel özetler

Bu alt sistemler birbirleriyle REST API üzerinden iletişim kurar. Tüm veri yönetimi MongoDB üzerinde sağlanır. Her alt sistem, kendi controller, model ve route dosyalarıyla ayrılmıştır; bu da mikro mimariye geçişi kolaylaştıracak bir

yapı sunar.

4. SINIF TANIMLARI

Bu bölümde, hayvan sahiplendirme platformunun temel veri yapıları (sınıflar) tanımlanmıştır. Sistem, NoSQL tabanlı MongoDB veritabanı kullandığı için sınıflar, Mongoose şemaları üzerinden modellenmiştir. Her sınıf, kendine özgü alanları (özellikleri) ve diğer sınıflarla ilişkileriyle birlikte tanımlanmıştır.

user.js

Platformun bireysel kullanıcılarını temsil eder.

Özellikler:

- * name, surname, username, email, password, phone, profileImage
- * roles, bio, links, followers, followings, bookmarks
- * isEmailVerified, createdAt, rates

petListing.js

Sahiplendirilmek istenen hayvanlara ait ilanları içerir.

Özellikler:

- * title, description, animalType, breed, age, gender, healthStatus
- * images, location, owner, isAdopted, createdAt

lostPetListing.js

Kayıp hayvan ilanlarını içerir.

Özellikler:

- * petName, description, lastSeenLocation, dateLost, contactInfo
- * images, reporter, isFound, createdAt

comment.js

İlanlara yapılan yorumları temsil eder.

Özellikler:

- * author, content, listingId, createdAt, likes, replies

commentReply.js

Yorumlara yapılan yanıtları içerir.

Özellikler:

- * parentComment, author, content, createdAt

notification.js

Kullanıcılara gösterilecek bildirimleri tutar.

Özellikler:

- * user, type, content, relatedListing, isRead, createdAt

report.js

Platformdaki kötüye kullanım şikayetlerini içerir.

Özellikler:

- * reporter, targetUser, reason, relatedListing, date, status

category.js ve subCategory.js

İlanların sınıflandırılmasını sağlar.

Özellikler:

- * name, slug, icon, parentCategory (alt kategoriler için)

auditlog.js

Yönetim veya sistem aktivitelerini kaydeder.

Özellikler:

- * action, user, target, timestamp, description, ip

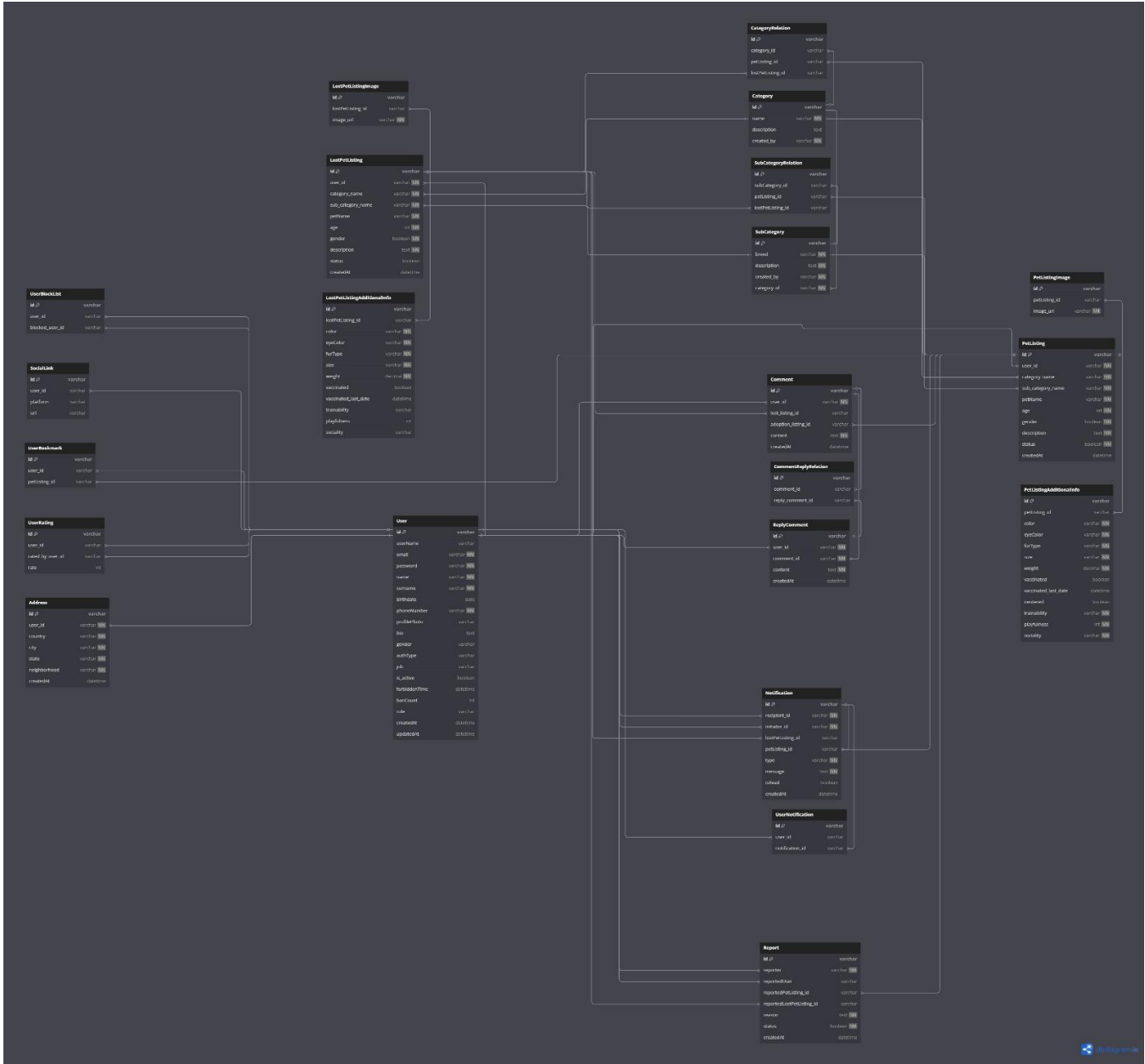
address.js

Kullanıcılara veya ilanlara ait adres bilgisini içerir.

Özellikler:

- * city, district, coordinates, fullAddress, zipCode

5- Veri Tabanı Mimarisi



6-Sistem Mimarisi: RESTful API Uç Noktaları

1. Kimlik Doğrulama (Auth)

* POST	/auth/signup	Yeni kullanıcı oluşturma
* POST	/auth/login	Oturum açma
* POST	/auth/logout	Oturum kapatma

2. Kullanıcı İşlemleri (Users)

* GET	/users	Platformun tüm kullanıcılarını getir
* GET	/users/:user_id	Belirli kullanıcıyı getir
* GET	/users/me	Kendi kullanıcı bilgilerini göster
* PUT	/users/me	Kendi kullanıcı bilgilerini güncelle
* GET	/users/me/notifications	Kullanıcının tüm bildirimlerini getir
* DELETE	/users/me/notifications/:notification_id	Belirli bildirimi sil
* PUT	/users/block/:user_id	Belirli kullanıcıyı engelle
* DELETE	/users/block/:user_id	Engeli kaldır
* GET	/users/me/block	Engellediği kullanıcıları getir
* POST	/users/report/:user_id	Kullanıcıyı şikayet et
* GET	/users/report/admin	Tüm şikayetleri getir
* PUT	/users/report/admin/:user_id	Kullanıcıyı banla
* GET	/users/me/listing	Kendi ilanlarını getir
* DELETE	/users/me/bookmarks/:listing_id	Belirli ilanı yer imlerinden sil
* GET	/users/me/bookmarks	Yer imlerini görüntüle
* GET	/users/:user_id/listing	Belirli kullanıcının ilanlarını getir

3. Kayıp Evcil Hayvan İlanları (Lost Pet Listings)

* GET	/lost_listing	Tüm kayıp ilanlarını getir
* POST	/lost_listing	Yeni kayıp ilanı oluştur
* GET	/lost_listing/:listing_id	Belirli kayıp ilanını getir
* PUT	/lost_listing/:listing_id	Belirli ilanı güncelle
* DELETE	/lost_listing/:listing_id	Belirli ilanı sil
* POST	/lost_listing/:listing_id/bookmarks	İlanı yer imlerine ekle
* GET	/lost_listing/:listing_id/comment	İlanın yorumlarını getir
* POST	/lost_listing/:listing_id/comment	İlana yorum yap
* PUT	/lost_listing/:listing_id/comment/:comment_id	Yorumu güncelle
* DELETE	/lost_listing/:listing_id/comment/:comment_id	Yorumu sil
* GET	/lost_listing/:listing_id/comment/:comment_id/reply_comment	Yorumun yanıtlarını getir
* POST	/lost_listing/:listing_id/comment/:comment_id/reply_comment	Yoruma yanıt yap
* PUT	/lost_listing/:listing_id/comment/:comment_id/reply_comment/:reply_id	Yanıtı güncelle
* DELETE	/lost_listing/:listing_id/comment/:comment_id/reply_comment/:reply_id	Yanıtı sil

4. Sahiplendirme İlanları (Adoption Listings)

* GET	/listing	Tüm ilanları getir
* POST	/listing	Yeni ilan oluştur
* GET	/listing/:listing_id	Belirli ilanı getir

* PUT	/listing/:listing_id	İlanı güncelle
* DELETE	/listing/:listing_id	İlanı sil
* POST	/listing/:listing_id/bookmarks	İlanı yer imlerine ekle
* GET	/listing/:listing_id/comment	İlanın yorumlarını getir
* POST	/listing/:listing_id/comment	İlana yorum yap
* GET	/listing/:listing_id/comment/:comment_id	Belirli yorumu getir
* PUT	/listing/:listing_id/comment/:comment_id	Yorumu güncelle
* DELETE	/listing/:listing_id/comment/:comment_id	Yorumu sil
* GET	/listing/:listing_id/comment/:comment_id/reply_comment	Yorumun yanıtlarını getir
* POST	/listing/:listing_id/comment/:comment_id/reply_comment	Yoruma yanıt yap
* PUT	/listing/:listing_id/comment/:comment_id/reply_comment/:reply_id	Yanıtı güncelle
* DELETE	/listing/:listing_id/comment/:comment_id/reply_comment/:reply_id	Yanıtı sil

5. Kategoriler ve Alt Kategoriler (Categories & Subcategories)

Kategoriler:

* GET	/categories	Tüm kategorileri listele
* POST	/categories	Yeni kategori oluştur
* PUT	/categories/:category_id	Kategoriye güncelle
* DELETE	/categories/:category_id	Kategoriye sil
* POST	/categories/:category_id/subcategories	Yeni alt kategori oluştur
*		

Alt Kategoriler:

* GET	/subcategories	Tüm alt kategorileri listele
* GET	/subcategories/:id	Alt kategori bilgisi getir
* PUT	/subcategories/:id	Alt kategoriye güncelle
* DELETE	/subcategories/:id	Alt kategoriye sil

6. Denetim Günlükleri (Audit Logs)

* GET	/auditlog	Tüm kullanıcı işlemlerini getir
-------	-----------	---------------------------------

7. Yönetici İşlemleri (Admin Operations)

* GET	/reports	Tüm şikayetleri getir
* PUT	/forbidden_user	Kullanıcıyı banla
* PUT	/changing_role	Kullanıcının rolünü değiştir

8. Arama (Search)

* GET	/search	Arama işlemi
-------	---------	--------------

7- Kullanılacak Teknolojiler ve Kodlama Standartları

Diller ve Frameworkler:

- * Frontend: Next.js, Tailwind CSS, Zustand.
- * Backend: Node.js, [Express.js](#), [Websocket.io](#) (MVP'den sonra)
- * Veritabanı: MongoDB
- * Diğer teknolojiler: Docker, JWT, Cloudinary

Kodlama Standartları:

- * İsimlendirme kuralları (camelCase, snake_case vs.)
- * Dosya yapısı
- * Kod yorumlama ve dökümantasyon kuralları
- * Linter kullanımı (ESLint gibi)

8- Test Yaklaşımları

- * Entegrasyon Testi: Alt sistemlerin birlikte çalışma testleri
- * Kabul Testi: Kullanıcı senaryoları üzerinden yapılan testler
- * Kullanılacak test araçları: ThunderClient, Postman, Lighthouse